

Day 2 - Memory Management

Games allocate and free constantly. Entities spawn and die, levels load and unload. Leaks and dangling pointers are the bugs that crash a game once things get heated. Today you learn where memory lives, who owns it, and how RAII makes cleanup automatic so you stop tracking `delete` by hand.

Learning objectives

By the end of today you can:

- [ ] Explain what lives on the stack vs the heap and the lifetime of each.
- [ ] Use pointers and references correctly and say when each is right.
- [ ] Pair every `new` with exactly one `delete` (and `new[]` with `delete[]`).
- [ ] Identify and fix a dangling pointer and a memory leak.
- [ ] Apply RAII so a resource is released by a destructor, not by hand.
- [ ] Choose the right smart pointer (`unique_ptr`, `shared_ptr`) and justify it.

Concepts

Stack vs heap

Automatic (stack) objects live until their enclosing scope ends and are freed for you. Dynamic (heap) objects live until *you* free them and can outlive the scope that made them. The stack is fast but small; reach for the heap when the size or lifetime isn't known at compile time.

```
void spawn(){
    Enemy local;                // stack: destroyed at the
closing brace
    Enemy* onHeap = new Enemy();  // heap: lives until you
delete it
    // ... if you forget `delete onHeap;`, it leaks
}
```

Pointers and references

A reference is an alias to an existing object: it can't be null and can't be reseated. A raw pointer can be null and repointed, useful for *observing*, but on its own it says nothing about who owns the pointee. Prefer references when the thing must exist; use a raw pointer only to observe, never to own.

```
void heal(Enemy& e);      // e must exist; not an owner
void heal(Enemy* e);      // e may be null; still not an owner,
just looks at it
```

❌ **Common Mistake:** treating a raw pointer as if holding it means owning it. Two "owners" then both `delete` it (double-free), or each assumes the other will (leak).

✅ **Good Practice:** let exactly one owner hold the resource (an RAII class or a smart pointer) and pass raw pointers only as non-owning observers that never `delete`.

new / delete

`new` allocates and constructs; `delete` destructs and frees. `new[]` must be paired with `delete[]`. Every path out of the scope, including early returns and exceptions, has to reach the matching `delete`, which is exactly why doing this by hand is fragile (and why RAII below exists).

```
int* scores = new int[8];
// ... use scores ...
delete[] scores;          // NOT plain `delete`: that is
undefined behaviour

Enemy* e = new Enemy();
delete e;                 // pairs with plain `new`
```

Dangling pointers & leaks

A **leak** is memory you allocated and never freed. A **dangling pointer** points at memory that's already been freed; reading/writing it is use-after-free, and freeing it again is a double-free. Both are undefined behaviour.

```
Enemy* e = new Enemy();
delete e;
e->update(16);   // use-after-free: e dangles
delete e;        // double-free
// (and if you never delete at all: a leak)
```

A leak detector reports the first as "definitely lost" bytes and the others as invalid reads/writes. Reproduce each once, on purpose, so you recognise the output.

RAII

Resource Acquisition Is Initialization: acquire a resource in a constructor and release it in the destructor, so cleanup happens automatically when the object leaves scope on every path, exceptions included. This is the idea the rest of the curriculum leans on.

```
class Buffer {
public:
    explicit Buffer(std::size_t n) : m_data(new int[n]) {}
    ~Buffer() { delete[] m_data; }   // freed automatically,
always

    Buffer(const Buffer&) = delete;             // forbid
copying: two owners would
    Buffer& operator=(const Buffer&) = delete;  //   double-free
(Day 3 covers this fully)
private:
    int* m_data;
};

void use(){
    Buffer b(1024);   // no manual delete anywhere; ~Buffer runs
at the brace
}
```

❌ **Common Mistake:** a RAII class with a destructor but no copy/move handling. The default copy duplicates the pointer, and then both destructors free it (double-free).

✅ **Good Practice:** whenever you write a destructor, delete or define copy *and* move alongside it (Day 3's rule of three/five).

Smart pointers

Standard-library RAII wrappers for ownership. `std::unique_ptr` is sole ownership (move-only, zero overhead); `std::shared_ptr` is shared, reference-counted ownership (use only when ownership is genuinely shared). A plain raw pointer is still fine for non-owning observation.

```
#include <memory>

auto e = std::make_unique<Enemy>();   // sole owner; freed when
`e` dies
Enemy* observer = e.get();            // observes, does not own:
never delete this
```

```
std::shared_ptr<World> w = std::make_shared<World>();
auto w2 = w;                            // two owners; freed when
the last one dies
```

❌ **Common Mistake:** `shared_ptr` as a reflex default. The reference count costs an atomic on every copy, and it hides who actually keeps the object alive.

✅ **Good Practice:** start with `unique_ptr` and reach for `shared_ptr` only when two owners must truly keep the object alive.

Running a leak detector

- **Linux:** `valgrind --leak-check=full ./a.out`
- **macOS:** `leaks --atExit -- ./a.out` (ships with Xcode's command-line tools).

Exercises

Start from this deliberately leaky program. Each exercise takes it a step further.

```
// starter.cpp: leaks on the early-return path, and never frees
on the normal one
#include <iostream>
struct Enemy { int hp = 100; };

int fight(bool flee){
    Enemy* e = new Enemy();
    if (flee) return -1;      // BUG: leaks e
    int result = e->hp;
    // BUG: no `delete e;` here either
    return result;
}
int main(){
    std::cout << fight(false) << ' ' << fight(true) << '\n';
}
```

ex00 - Manual then leak-free

.	.	.
💢	**Difficulty**	⭐⭐
🧠	**Objectives**	*Fix a leaky function so both the early return and the normal return reach exactly one `delete`, with no behaviour change.*
📂	**Turn in directory**	`ex00/`
❗	**Turn in files**	**main.cpp, Makefile**
🛠️	**Skills required**	*Pair every `new` with one `delete`, trace all exit paths of a function, run a leak detector.*
🔍	**Search keywords**	*new delete C++, memory leak, early return leak, valgrind leak-check full, leaks --atExit macOS*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`

- **Goal:** Fix the starter so `Enemy` is freed on *every* path: the early return and the normal return both reach exactly one `delete`. No behaviour change.
- **Done when:** The leak detector reports 0 bytes leaked for both `fight(true)` and `fight(false)`.

ex01 - Make it RAII

.	.	.
💢	**Difficulty**	⭐⭐⭐
🧠	**Objectives**	*Move an object's lifetime out of `fight` and into a small RAII wrapper (call it `Owned`) so allocation and its single matching free live inside that one type.*
📂	**Turn in directory**	`ex01/`
❗	**Turn in files**	**Owned.hpp, Owned.cpp, main.cpp, Makefile**
🛠️	**Skills required**	*RAII (acquire in a constructor, release in the destructor), giving natural access to the owned object, deleting or defining copy so one object is never freed twice.*
🔍	**Search keywords**	*RAII C++, resource acquisition is initialization, destructor cleanup, deleted copy constructor, rule of three, operator-> overload*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`

- **Start from:** A leak-free `fight` from the previous exercise. Copy it into `ex01/` as your baseline. You will now be changing how `Enemy` is owned.
- **Goal:** Move responsibility for the `Enemy`'s lifetime out of `fight` and into a small type of your own (call it `Owned`) whose job is to allocate and free the `Enemy` automatically when it goes out of scope. When you're done, `fight` gets its `Enemy` through that type and never frees anything by hand.
- **Requirements:**
  - The allocation and its one matching free both live inside `Owned`. Nothing anywhere else in the program calls `delete`.
  - Code can still reach the `Enemy` through an `Owned` as naturally as it reached it through the raw pointer.
  - Decide what it should *mean* to copy an `Owned`, then make the compiler hold you to that decision. A copy must never let one `Enemy` be freed twice.
- **Done when:** No `delete` appears outside `Owned`, `make` builds clean under the mandatory flags, and the leak detector reports 0 bytes for both `fight(true)` and `fight(false)`.

ex02 - Pick the pointer

.	.	.
💢	**Difficulty**	⭐⭐⭐
🧠	**Objectives**	*Replace the hand-written wrapper with the correct standard smart pointer, justify the choice, and add a non-owning raw pointer that observes the object without freeing it.*
📁	**Turn in directory**	`ex02/`
❗	**Turn in files**	**main.cpp, Makefile**
🛠️	**Skills required**	*Choose `unique_ptr` vs `shared_ptr`, create with `make_unique` / `make_shared`, distinguish an owning pointer from a non-owning (observing) one.*
🔍	**Search keywords**	*std::unique_ptr, std::shared_ptr, std::make_unique, smart pointer ownership, get() non-owning raw pointer, move-only type*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<memory>`

- **Start from:** Your ex01 solution. Copy `main.cpp` and the `Makefile` into `ex02/`, and leave `Owned.hpp` / `Owned.cpp` behind. The standard library replaces the type you hand-wrote.
- **Goal:** Own the `Enemy` in `fight` with a standard smart pointer instead of `Owned`. Same behaviour, same output, and the whole wrapper you wrote before disappears.
- **Requirements:**
  - `fight(bool flee)` and `main` keep the signatures and behaviour they have in the starter.
  - Create the `Enemy` with `std::make_unique` or `std::make_shared`. Pick one and justify it in a one-line comment above that line.
  - Read the hp through a non-owning raw pointer taken from `.get()`, not through the smart pointer directly. That pointer never gets `delete`d.
- **Done when:** `Owned` is gone, `new` and `delete` appear nowhere in `main.cpp`, `./fight` prints the output below, and the leak detector reports 0 bytes for both `fight(true)` and `fight(false)`.

Output example:

```
$ ./fight
```

```
100 -1
```

Identical to ex01's output. You deleted a constructor, a destructor, and the copy control you wrote by hand, and the leak guarantee held anyway.

Constraints (every exercise)

- Compiles with `c++ -Wall -Wextra -Werror -std=c++17`.
- **Zero leaks**, checked under a leak detector (`valgrind` on Linux, `leaks --atExit` on macOS).
- No raw owning pointers in ex02.
- Follows Day 1 naming and header conventions.

How you're evaluated

An evaluation will be automatically scheduled with a peer and start at 6pm. Submit all the work you have done so far even if its not 100% complete.

Turning in your work

1. Create a new repository on GitHub named `gauntlet-day-02`, and keep it **public**. A private repo is one your evaluator cannot clone, and the evaluation cannot start without the code.
2. Commit today's work there, one directory per exercise (`ex00/`, `ex01/`, ...) exactly as each turn-in table says, `Makefile` included.
3. Push before the evaluation. Whatever sits on the default branch when the evaluation starts is what gets read.
4. When the evaluation starts, give your evaluator the link to the repository.

AI-written code is an immediate disqualification. Look things up and ask an AI what something does, that is a tool. Pasting in code you did not write is not.